

Project 4A

Wokwi Simulated Hardware Test Bench

Full Complete Build Guide, Step-by-Step Explanation, Commands, Evidence, GitHub Workflow, and Troubleshooting

Owner	Aaron Trujillo
GitHub Repo	https://github.com/aarontrujillofz-ee/project-4a-wokwi-simulated-hardware-test-bench
Project Type	Simulated embedded hardware validation / hardware test portfolio project
Main Tools	Wokwi, Arduino Uno simulation, Python, pandas, matplotlib, reportlab, PowerShell, VS Code, Git, GitHub
Final Result	FAIL - the simulated signal crossed the 4.0 V failure threshold

This document explains exactly what was built, why it matters, how each file works, how to reproduce the project, how to run the analysis, how to save evidence, and how the project was uploaded to GitHub.

Table of Contents

- 1. Project summary
- 2. Engineering concept explained
- 3. Software stack and folder structure
- 4. Step-by-step build process
- 5. Wokwi simulation workflow
- 6. Python analysis workflow
- 7. Evidence and final results
- 8. README and documentation
- 9. Git and GitHub upload
- 10. Troubleshooting
- 11. How to explain the project in interviews
- 12. Resume bullet
- 13. Next upgrade: Project 4B
- Appendix A: Key files and code

1. Project Summary

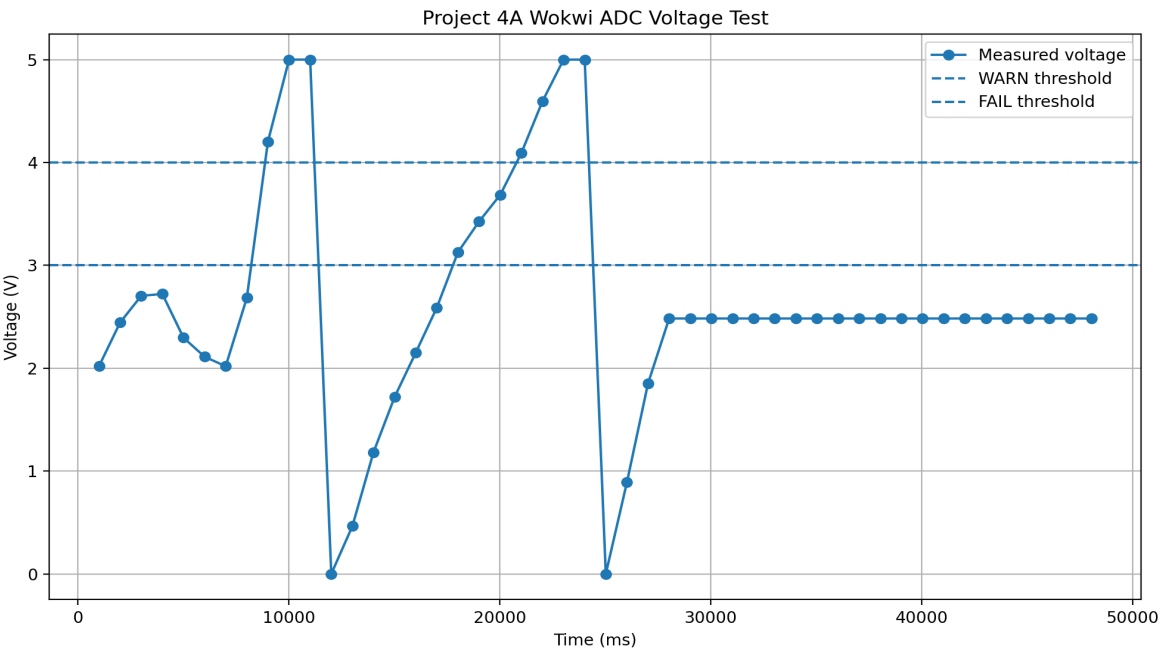
Project 4A is a simulated embedded hardware validation project. It uses Wokwi to simulate an Arduino Uno reading an analog voltage from a potentiometer. The Arduino code converts ADC counts into voltage, labels each sample as NORMAL, WARN, or FAIL, and prints CSV-style data through the Serial Monitor. Python then reads the CSV, cleans it, calculates statistics, creates a voltage plot, writes a text summary, and generates a PDF validation report.

The project is complete because it includes working simulation files, real Wokwi-generated CSV data, Python analysis, plots, reports, evidence files, README documentation, a clean Git commit, and a public GitHub repository.

Final result snapshot

Metric	Final Result
Total samples	48
ADC range	0 to 1023
Voltage range	0.000 V to 5.000 V
Average voltage	2.607 V
NORMAL readings	38
WARN readings	3
FAIL readings	7
Final result	FAIL

The final result is FAIL because the data includes samples at or above the 4.0 V failure threshold. That is not a mistake. It proves the validation logic correctly catches out-of-limit conditions.



Final voltage plot from the Wokwi potentiometer validation run.

2. Engineering Concept Explained

The complete validation pipeline

```

Simulated analog input
-> Arduino ADC reading
-> Voltage conversion
-> NORMAL / WARN / FAIL classification
-> Serial CSV output
-> Python data analysis
-> Voltage plot
-> Text summary
-> Automated PDF report
-> Evidence saved
-> GitHub documentation
  
```

This is the same basic thinking used in hardware test and validation roles: define limits, collect data, process measurements, classify the result, and preserve evidence.

What the potentiometer represents

A potentiometer is used as a controllable analog voltage source. Turning the knob changes the voltage at Arduino analog input A0. This is not a real sensor yet, but it is the cleanest first test input because it lets you sweep from low voltage to high voltage and prove the data pipeline works before adding sensor math.

ADC conversion

The simulated Arduino Uno ADC produces values from 0 to 1023. The project assumes a 5.0 V reference, so the voltage is calculated as:

```

voltage = adc_value * 5.0 / 1023.0
  
```

ADC Value	Approx. Voltage	Meaning
0	0.000 V	Minimum input
512	2.502 V	About half-scale
1023	5.000 V	Maximum input

Pass / warn / fail logic

Status	Condition	Meaning
NORMAL	voltage < 3.0 V	Signal is inside normal range
WARN	3.0 V <= voltage < 4.0 V	Signal is approaching failure threshold
FAIL	voltage >= 4.0 V	Signal crossed failure threshold

The final run result uses highest severity. If even one FAIL sample appears, the final result is FAIL. If there are no FAIL samples but at least one WARN sample appears, the final result is WARN. If all samples are normal, the final result is NORMAL.

3. Software Stack and What Each Tool Did

Tool	Role in the project
Wokwi	Browser-based simulator used to create the Arduino Uno + potentiometer circuit and Serial Monitor output.
Arduino Uno simulation	Microcontroller model used to read analog pin A0.
Potentiometer	Simulated analog input device.

Tool	Role in the project
VS Code	Project editor used to inspect README, code, .gitignore, and project files.
PowerShell	Command-line shell used to create folders, activate Python, run scripts, and use Git.
Python	Data analysis and reporting language.
pandas	Reads and cleans CSV data.
matplotlib	Generates the voltage plot.
reportlab	Generates the PDF report.
Git	Tracks local project history and creates commits.
GitHub	Hosts the public portfolio repository.

Final folder structure

Project_4A_Wokwi_Simulated_Hardware_Test_Bench/

```
|
|-- README.md
|-- requirements.txt
|-- .gitignore
|-- .gitattributes
|
|-- wokwi/
|   |-- sketch.ino
|   |-- diagram.json
|
|-- python/
|   |-- analyze_wokwi_log.py
|
|-- data/
|   |-- sample_wokwi_serial_log.csv
|   |-- wokwi_serial_log.csv
|
|-- plots/
|   |-- wokwi_voltage_plot.png
|
|-- reports/
|   |-- project4a_summary.txt
|   |-- project4a_wokwi_test_report.pdf
|
|-- docs/
|   |-- test_plan.md
|   |-- engineering_notes.md
|
|-- evidence/
|   |-- wokwi_voltage_plot_stress_test.png
|   |-- project4a_summary_stress_test.txt
|   |-- project4a_report_stress_test.pdf
|   |-- wokwi_voltage_plot_final_run.png
|   |-- project4a_summary_final_run.txt
|   |-- project4a_report_final_run.pdf
|
|-- screenshots/
```

The .venv folder was kept locally and correctly ignored by Git. That matters because virtual environments are large, machine-specific, and should not be uploaded to GitHub.

4. Step-by-Step Build Process

Step 1 - Create the project folder

The project was created on the Desktop inside a dedicated folder. This kept the project files separate from other work.

```
cd "C:\Users\fall3\Desktop"
```

```
mkdir Project_4A_Wokwi_Simulated_Hardware_Test_Bench
cd "C:\Users\fall3\Desktop\Project_4A_Wokwi_Simulated_Hardware_Test_Bench"
```

Step 2 - Create the folder structure

The folders separated code, data, plots, reports, documentation, Wokwi files, and evidence. This makes the repo easier for recruiters to scan.

```
mkdir wokwi
mkdir python
mkdir data
mkdir plots
mkdir reports
mkdir docs
mkdir evidence
mkdir photos
mkdir screenshots
```

Step 3 - Create requirements.txt

requirements.txt tells another user which Python packages are needed to run the project.

```
pandas
matplotlib
reportlab
```

Step 4 - Create .gitignore

.gitignore prevents junk files and the local Python virtual environment from going to GitHub.

```
.venv/
__pycache__/
*.pyc
.DS_Store
Thumbs.db
```

Step 5 - Create .gitattributes

.gitattributes was added to treat PDFs and PNG files as binary files and text/code files as text. This reduces Git line-ending problems for evidence files.

```
*.pdf binary
*.png binary
*.jpg binary
*.jpeg binary

*.md text
*.txt text
*.csv text
*.py text
*.ino text
*.json text
```

5. Wokwi Simulation Workflow

What sketch.ino does

sketch.ino is the Arduino program. It reads A0, converts the ADC count to voltage, classifies the voltage, and prints CSV data.

- sensorPin = A0 selects the analog input.
- adcToVoltage() converts raw ADC counts into volts.
- classifyVoltage() returns NORMAL, WARN, or FAIL.

- `setup()` starts Serial at 9600 baud and prints the CSV header.
- `loop()` reads the ADC once per second and prints one CSV row.

What diagram.json does

`diagram.json` tells Wokwi which parts exist and how they are wired. In this project, it places an Arduino Uno and a potentiometer, then connects the potentiometer signal pin to A0, VCC to 5V, and GND to GND.

```
{
  "version": 1,
  "author": "Aaron Trujillo",
  "editor": "wokwi",
  "parts": [
    { "type": "wokwi-arduino-uno", "id": "uno", "top": 0, "left": 0, "attrs": {} },
    { "type": "wokwi-potentiometer", "id": "pot1", "top": 140, "left": 230, "attrs": {} }
  ],
  "connections": [
    [ "pot1:SIG", "uno:A0", "green", [ "v0" ] ],
    [ "pot1:VCC", "uno:5V", "red", [ "v0" ] ],
    [ "pot1:GND", "uno:GND.1", "black", [ "v0" ] ]
  ],
  "dependencies": {}
}
```

Running the Wokwi simulation

In Wokwi, the process was:

- Create a new Arduino Uno project.
- Paste the local `sketch.ino` code into Wokwi `sketch.ino`.
- Paste the local `diagram.json` into Wokwi `diagram.json`.
- Click Run.
- Move the potentiometer knob to create different ADC values.
- Copy the Serial Monitor output.
- Save that output into `data/wokwi_serial_log.csv`.

Serial output format

```
time_ms,adc_value,voltage,status
999,413,2.019,NORMAL
9007,860,4.203,FAIL
18016,640,3.128,WARN
```

Column	Meaning
<code>time_ms</code>	Time since the Wokwi simulation started, in milliseconds.
<code>adc_value</code>	Raw Arduino ADC count from 0 to 1023.
<code>voltage</code>	Calculated voltage from the ADC count.
<code>status</code>	NORMAL, WARN, or FAIL based on threshold logic.

6. Python Analysis Workflow

Create and activate the virtual environment

The virtual environment keeps project packages isolated from the rest of the computer. The prompt should show (.venv) when it is active.

```
py -m venv .venv
Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass
.\.venv\Scripts\Activate.ps1
```

Install dependencies

```
pip install -r requirements.txt
```

Run the analyzer

```
python .\python\analyze_wokwi_log.py
```

What analyze_wokwi_log.py does

- Chooses data/wokwi_serial_log.csv if it exists.
- Falls back to sample_wokwi_serial_log.csv only if real Wokwi data is missing.
- Reads the CSV using pandas.
- Removes duplicate header rows caused by restarting Wokwi.
- Converts time, ADC, and voltage columns into numeric values.
- Keeps only valid NORMAL, WARN, and FAIL rows.
- Calculates sample count, min/max ADC, min/max voltage, average voltage, status counts, and final result.
- Creates plots/wokwi_voltage_plot.png.
- Creates reports/project4a_summary.txt.
- Creates reports/project4a_wokwi_test_report.pdf.

Expected successful output

```
Project 4A Wokwi Simulated Hardware Test Bench
-----
Data file used: ...data\wokwi_serial_log.csv
Total samples: 48
Min ADC: 0
Max ADC: 1023
Min voltage: 0.000 V
Max voltage: 5.000 V
Average voltage: 2.607 V
Status counts: {'NORMAL': 38, 'FAIL': 7, 'WARN': 3}
Final result: FAIL
Saved plot to: ...plots\wokwi_voltage_plot.png
Saved summary to: ...
eports\project4a_summary.txt
Saved PDF report to: ...
eports\project4a_wokwi_test_report.pdf
```

7. Evidence and Final Results

Generated evidence files

File	Purpose
plots/wokwi_voltage_plot.png	Main voltage plot from the analysis.

File	Purpose
reports/project4a_summary.txt	Text summary of sample count, ADC range, voltage range, counts, and final result.
reports/project4a_wokwi_test_report.pdf	Generated PDF engineering report.
evidence/project4a_report_final_run.pdf	Archived final run report.
evidence/project4a_summary_final_run.txt	Archived final run summary.
evidence/wokwi_voltage_plot_final_run.png	Archived final run plot.
evidence/project4a_report_stress_test.pdf	Archived stress test report.

Commands used to save evidence

```
mkdir evidence -Force
copy .\plots\wokwi_voltage_plot.png .\evidence\wokwi_voltage_plot_final_run.png
copy .\reports\project4a_summary.txt .\evidence\project4a_summary_final_run.txt
copy .\reports\project4a_wokwi_test_report.pdf .\evidence\project4a_report_final_run.pdf

dir .\evidence
```

Interpretation of the final run

The final run is a FAIL because seven samples crossed the failure threshold. This is useful evidence: the project did not merely generate a graph; it proved the logic can catch out-of-limit measurements.

8. README and Documentation

The README explains what the project is, why it matters, how the circuit works, how to run the Python analyzer, how to use the Wokwi files, what the final results were, what the limitations are, and how this project should be discussed in interviews.

Good README traits

- Honest: it says this is simulated hardware, not real hardware.
- Clear: it explains the full validation workflow.
- Evidence-based: it includes final metrics and generated output files.
- Recruiter-friendly: it has a resume bullet and interview explanation.
- Future-oriented: it points to Project 4B and real-hardware upgrades.

Docs folder

The docs folder contains test_plan.md and engineering_notes.md. These documents make the project feel more like engineering work and less like a random script.

Document	Purpose
test_plan.md	Defines objective, input, thresholds, required evidence, and upgrade path.
engineering_notes.md	Explains why the potentiometer was used, the ADC formula, validation meaning, and limitations.

9. Git and GitHub Upload

Local Git setup

```
git init
git status
git add .
git status
git commit -m "Add Project 4A Wokwi simulated hardware test bench"
git branch -M main
git status
```

What each Git command did

Command	Explanation
git init	Creates a local Git repository.
git status	Shows what files are tracked, untracked, staged, or clean.
git add .	Stages the project files for commit.
git commit -m "..."	Saves a snapshot of the project locally.
git branch -M main	Renames the branch from master to main.
git remote add origin	Connects the local repo to the GitHub repo.
git push -u origin main	Uploads the local main branch to GitHub and tracks it.

GitHub repository settings

Setting	Used Value
Repository name	project-4a-wokwi-simulated-hardware-test-bench
Visibility	Public
Add README	Off
Add .gitignore	No .gitignore
Add license	No license

Push commands

```
git remote add origin https://github.com/aarontrujillofz-ee/project-4a-wokwi-simulated-hardware-test-bench.git
git push -u origin main
```

Final clean status

On branch main
Your branch is up to date with 'origin/main'.

nothing to commit, working tree clean

That final status means the local project and GitHub repo are synced. This is the clean end state.

10. Troubleshooting

Problem	Cause	Fix
PowerShell says a path is not recognized	You typed a folder path like a command.	Use cd "full path" first.

Problem	Cause	Fix
copy cannot find plots or reports	You are in the wrong folder.	cd into Project_4A_Wokwi_Simulated_Hardware_Test_Bench before copying.
ModuleNotFoundError: reportlab	The virtual environment is not activated.	Run <code>.\venv\Scripts\Activate.ps1</code> , then rerun Python.
.venv appears in git status	The .gitignore is wrong or was added too late.	Fix .gitignore, then use <code>git rm --cached -r .</code> and <code>git add .</code> again.
LF will be replaced by CRLF warnings	Windows line-ending warning.	Usually safe. .gitattributes helps reduce binary-file problems.
Wokwi output has repeated headers	Simulation restarted and printed the header again.	Python script removes duplicate headers, or manually clean the CSV.
GitHub merge problem	You created README on GitHub and locally.	Do not add a GitHub README when you already have one locally.
remote origin already exists	Remote was already configured.	Use <code>git remote set-url origin .</code>

11. How to Explain the Project in Interviews

Strong explanation:

For Project 4A, I built a simulated hardware test bench in Wokwi using an Arduino Uno and potentiometer. The Arduino reads analog voltage on A0, converts ADC counts into voltage, classifies each reading as NORMAL, WARN, or FAIL, and prints CSV-style serial data. I then process that data in Python to generate statistics, plots, and a PDF engineering report. This project proves the validation pipeline before moving to real hardware.

What this says about you

- You understand raw data collection.
- You can convert measurements into engineering units.
- You can define and apply threshold logic.
- You can automate reporting with Python.
- You can document engineering evidence.
- You can use Git and GitHub to publish professional work.

12. Resume Bullet

Use this bullet for hardware test, validation, embedded test, engineering technician, or entry-level electrical engineering roles:

Built a Wokwi-based simulated embedded hardware test bench using Arduino Uno ADC logic and Python analysis to collect potentiometer-based analog readings, convert ADC counts to voltage, evaluate NORMAL/WARN/FAIL thresholds, generate CSV data, create voltage plots, and produce automated PDF validation reports.

13. Next Upgrade: Project 4B

Project 4A is done. The correct next move is not endless polishing. The correct next move is Project 4B: Wokwi NTC Thermistor Temperature Test Bench.

Why Project 4B is stronger

- It connects directly to the earlier NTC thermistor voltage divider project.
- It replaces a generic knob with a sensor-like temperature workflow.
- It adds temperature conversion logic, not just voltage conversion.
- It looks closer to embedded sensor validation and hardware test work.
- It prepares the path toward a real Arduino or Tiva-C hardware build.

Project 4B high-level workflow

Simulated thermistor or temperature sensor
-> Arduino ADC reading
-> Voltage conversion
-> Resistance or temperature calculation
-> NORMAL / WARN / FAIL classification
-> Serial CSV output
-> Python analysis
-> Temperature plot
-> PDF validation report
-> GitHub repo update or new repo

Appendix A: Key Files and Code

A1. sketch.ino

```
■const int sensorPin = A0;

float adcToVoltage(int adcValue) {
    return adcValue * (5.0 / 1023.0);
}

String classifyVoltage(float voltage) {
    if (voltage >= 4.0) {
        return "FAIL";
    }
    if (voltage >= 3.0) {
        return "WARN";
    }
    return "NORMAL";
}

void setup() {
    Serial.begin(9600);
    delay(1000);
    Serial.println("time_ms,adc_value,voltage,status");
}

void loop() {
    unsigned long timeMs = millis();
    int adcValue = analogRead(sensorPin);
    float voltage = adcToVoltage(adcValue);
    String status = classifyVoltage(voltage);

    Serial.print(timeMs);
    Serial.print(",");
    Serial.print(adcValue);
    Serial.print(",");
    Serial.print(voltage, 3);
    Serial.print(",");
    Serial.println(status);

    delay(1000);
}
```

A2. diagram.json

```

■{
  "version": 1,
  "author": "Aaron Trujillo",
  "editor": "wokwi",
  "parts": [
    { "type": "wokwi-arduino-uno", "id": "uno", "top": 0, "left": 0, "attrs": {} },
    { "type": "wokwi-potentiometer", "id": "pot1", "top": 140, "left": 230, "attrs": {} }
  ],
  "connections": [
    [ "pot1:SIG", "uno:A0", "green", [ "v0" ] ],
    [ "pot1:VCC", "uno:5V", "red", [ "v0" ] ],
    [ "pot1:GND", "uno:GND.1", "black", [ "v0" ] ]
  ],
  "dependencies": {}
}

```

A3. analyze_wokwi_log.py

This is the main Python analysis file. It is included here because it is the most important proof of automation in the project.

```

■import sys
from pathlib import Path

import pandas as pd
import matplotlib.pyplot as plt
from reportlab.lib.pagesizes import letter
from reportlab.pdfgen import canvas

# =====
# Project 4A - Wokwi Simulated Hardware Test Bench
# Reads Wokwi Serial Monitor CSV data, cleans it, analyzes it,
# creates a voltage plot, summary text file, and PDF report.
# =====

ROOT = Path(__file__).resolve().parents[1]

REAL_WOKWI_FILE = ROOT / "data" / "wokwi_serial_log.csv"
SAMPLE_FILE = ROOT / "data" / "sample_wokwi_serial_log.csv"

PLOTS_DIR = ROOT / "plots"
REPORTS_DIR = ROOT / "reports"

PLOTS_DIR.mkdir(exist_ok=True)
REPORTS_DIR.mkdir(exist_ok=True)

def choose_data_file() -> Path:
    """
    Use real Wokwi data if it exists.
    Fall back to sample data only if real Wokwi data is missing.
    """
    if REAL_WOKWI_FILE.exists():
        return REAL_WOKWI_FILE

    if SAMPLE_FILE.exists():
        print("WARNING: data/wokwi_serial_log.csv was not found.")
        print("Using sample_wokwi_serial_log.csv instead.")
        return SAMPLE_FILE

    print("ERROR: No CSV data file found.")
    print(f"Expected: {REAL_WOKWI_FILE}")
    print(f"Or sample: {SAMPLE_FILE}")
    sys.exit(1)

def load_and_clean_data(data_file: Path) -> pd.DataFrame:
    """
    Loads Wokwi CSV data and cleans common Serial Monitor problems:
    - duplicate header rows

```

```

- blank rows
- bad numeric rows
- extra spaces
"""

df = pd.read_csv(data_file, dtype=str)

required_columns = {"time_ms", "adc_value", "voltage", "status"}
actual_columns = set(df.columns)

if not required_columns.issubset(actual_columns):
    print("ERROR: CSV file does not have the correct header.")
    print("The first line must be:")
    print("time_ms,adc_value,voltage,status")
    print(f"Current columns found: {list(df.columns)}")
    sys.exit(1)

# Remove duplicate header rows caused by restarting Wokwi simulation
df = df[df["time_ms"].str.strip() != "time_ms"].copy()

# Clean text
df["status"] = df["status"].astype(str).str.strip().str.upper()

# Convert numeric columns
df["time_ms"] = pd.to_numeric(df["time_ms"], errors="coerce")
df["adc_value"] = pd.to_numeric(df["adc_value"], errors="coerce")
df["voltage"] = pd.to_numeric(df["voltage"], errors="coerce")

# Drop bad rows
df = df.dropna(subset=["time_ms", "adc_value", "voltage", "status"])

# Keep only valid status values
df = df[df["status"].isin(["NORMAL", "WARN", "FAIL"]).copy()]

# Sort by time
df = df.sort_values("time_ms").reset_index(drop=True)

if df.empty:
    print("ERROR: CSV file loaded, but no valid data rows were found.")
    print("Check your Wokwi Serial Monitor output.")
    sys.exit(1)

return df

def determine_final_result(status_counts: dict) -> str:
    """
    Final result uses highest severity found in the whole run.
    """
    if status_counts.get("FAIL", 0) > 0:
        return "FAIL"
    if status_counts.get("WARN", 0) > 0:
        return "WARN"
    return "NORMAL"

def create_voltage_plot(df: pd.DataFrame) -> Path:
    """
    Creates voltage plot with WARN and FAIL thresholds.
    """

    plot_path = PLOTS_DIR / "wokwi_voltage_plot.png"

    plt.figure(figsize=(10, 6))
    plt.plot(df["time_ms"], df["voltage"], marker="o", label="Measured voltage")
    plt.axhline(y=3.0, linestyle="--", label="WARN threshold")
    plt.axhline(y=4.0, linestyle="--", label="FAIL threshold")
    plt.xlabel("Time (ms)")
    plt.ylabel("Voltage (V)")
    plt.title("Project 4A Wokwi ADC Voltage Test")
    plt.legend()
    plt.grid(True)
    plt.tight_layout()
    plt.savefig(plot_path, dpi=200)
    plt.close()

```

```

    return plot_path

def create_summary_text(
    data_file: Path,
    total_samples: int,
    min_adc: int,
    max_adc: int,
    min_voltage: float,
    max_voltage: float,
    avg_voltage: float,
    status_counts: dict,
    final_result: str,
) -> Path:
    """
    Creates text summary report.
    """

    summary_path = REPORTS_DIR / "project4a_summary.txt"

    with open(summary_path, "w", encoding="utf-8") as f:
        f.write("Project 4A Wokwi Simulated Hardware Test Bench\n")
        f.write("-----\n")
        f.write(f>Data file used: {data_file.name}\n")
        f.write(f>Total samples: {total_samples}\n")
        f.write(f>Min ADC: {min_adc}\n")
        f.write(f>Max ADC: {max_adc}\n")
        f.write(f>Min voltage: {min_voltage:.3f} V\n")
        f.write(f>Max voltage: {max_voltage:.3f} V\n")
        f.write(f>Average voltage: {avg_voltage:.3f} V\n")
        f.write(f>Status counts: {status_counts}\n")
        f.write(f>Final result: {final_result}\n")

    return summary_path

def create_pdf_report(
    data_file: Path,
    plot_path: Path,
    total_samples: int,
    min_adc: int,
    max_adc: int,
    min_voltage: float,
    max_voltage: float,
    avg_voltage: float,
    status_counts: dict,
    final_result: str,
) -> Path:
    """
    Creates PDF engineering report.
    """

    pdf_path = REPORTS_DIR / "project4a_wokwi_test_report.pdf"

    c = canvas.Canvas(str(pdf_path), pagesize=letter)
    width, height = letter

    y = height - 60

    c.setFont("Helvetica-Bold", 16)
    c.drawString(50, y, "Project 4A - Wokwi Simulated Hardware Test Bench")

    y -= 30
    c.setFont("Helvetica", 10)
    c.drawString(
        50,
        y,
        "Purpose: Simulate a microcontroller ADC test workflow using Wokwi and Python analysis.",
    )

    y -= 25
    c.drawString(50, y, f>Data file used: {data_file.name}")

    y -= 35
    c.setFont("Helvetica-Bold", 12)

```

```

c.drawString(50, y, "Engineering Summary")

y -= 20
c.setFont("Helvetica", 10)

lines = [
    f"Total samples: {total_samples}",
    f"ADC range: {min_adc} to {max_adc}",
    f"Voltage range: {min_voltage:.3f} V to {max_voltage:.3f} V",
    f"Average voltage: {avg_voltage:.3f} V",
    f"Status counts: {status_counts}",
    f"Final result: {final_result}",
]

for line in lines:
    c.drawString(70, y, line)
    y -= 16

y -= 15
c.setFont("Helvetica-Bold", 12)
c.drawString(50, y, "Pass / Warn / Fail Logic")

y -= 20
c.setFont("Helvetica", 10)

logic_lines = [
    "NORMAL: voltage < 3.0 V",
    "WARN: 3.0 V <= voltage < 4.0 V",
    "FAIL: voltage >= 4.0 V",
]

for line in logic_lines:
    c.drawString(70, y, line)
    y -= 16

y -= 20
c.setFont("Helvetica-Bold", 12)
c.drawString(50, y, "Conclusion")

y -= 20
c.setFont("Helvetica", 10)
c.drawString(
    70,
    y,
    f"The simulated ADC workflow completed with final result: {final_result}.",
)
y -= 16
c.drawString(
    70,
    y,
    "This proves the data path from analog-style input to ADC conversion, classification,",
)
y -= 16
c.drawString(
    70,
    y,
    "CSV logging, plotting, and automated engineering reporting.",
)

# Add plot on second page
c.showPage()
c.setFont("Helvetica-Bold", 14)
c.drawString(50, height - 60, "Voltage Plot")

if plot_path.exists():
    c.drawImage(
        str(plot_path),
        50,
        170,
        width=500,
        height=300,
        preserveAspectRatio=True,
        anchor="c",
    )

```

```

c.save()

return pdf_path

def main() -> None:
    data_file = choose_data_file()
    df = load_and_clean_data(data_file)

    total_samples = len(df)
    min_adc = int(df["adc_value"].min())
    max_adc = int(df["adc_value"].max())
    min_voltage = float(df["voltage"].min())
    max_voltage = float(df["voltage"].max())
    avg_voltage = float(df["voltage"].mean())

    status_counts = df["status"].value_counts().to_dict()
    final_result = determine_final_result(status_counts)

    print("Project 4A Wokwi Simulated Hardware Test Bench")
    print("-----")
    print(f"Data file used: {data_file}")
    print(f"Total samples: {total_samples}")
    print(f"Min ADC: {min_adc}")
    print(f"Max ADC: {max_adc}")
    print(f"Min voltage: {min_voltage:.3f} V")
    print(f"Max voltage: {max_voltage:.3f} V")
    print(f"Average voltage: {avg_voltage:.3f} V")
    print(f"Status counts: {status_counts}")
    print(f"Final result: {final_result}")

    plot_path = create_voltage_plot(df)

    summary_path = create_summary_text(
        data_file=data_file,
        total_samples=total_samples,
        min_adc=min_adc,
        max_adc=max_adc,
        min_voltage=min_voltage,
        max_voltage=max_voltage,
        avg_voltage=avg_voltage,
        status_counts=status_counts,
        final_result=final_result,
    )

    pdf_path = create_pdf_report(
        data_file=data_file,
        plot_path=plot_path,
        total_samples=total_samples,
        min_adc=min_adc,
        max_adc=max_adc,
        min_voltage=min_voltage,
        max_voltage=max_voltage,
        avg_voltage=avg_voltage,
        status_counts=status_counts,
        final_result=final_result,
    )

    print(f"Saved plot to: {plot_path}")
    print(f"Saved summary to: {summary_path}")
    print(f"Saved PDF report to: {pdf_path}")

if __name__ == "__main__":
    main()

```

A4. test_plan.md

■# Test Plan - Project 4A Wokwi Simulated Hardware Test Bench

Objective

Verify that a simulated Arduino ADC circuit can read analog input, convert ADC counts to voltage, classify the result, and provide data for Python analysis.

Test Input

Potentiometer connected to Arduino Uno analog pin A0.

```
## Pass / Warn / Fail Logic
- NORMAL: voltage < 3.0 V
- WARN: 3.0 V <= voltage < 4.0 V
- FAIL: voltage >= 4.0 V

## Evidence Required
- Wokwi circuit screenshot
- Serial monitor screenshot
- CSV log
- Python terminal summary
- Voltage plot
- PDF report
- README documentation

## Upgrade Path
Replace potentiometer with:
1. NTC thermistor
2. LM35 or TMP36 temperature sensor
3. Real Arduino hardware
4. Tiva-C TM4C123GXL hardware
```

A5. engineering_notes.md

■# Engineering Notes

Why Start with a Potentiometer?

A potentiometer is the cleanest first ADC test input. It lets the user manually sweep voltage from low to high and verify ADC conversion before adding sensor math.

ADC Formula
For Arduino Uno:

```
voltage = adc_value * 5.0 / 1023.0
```

Validation Meaning
This project proves the test workflow:

input signal -> ADC reading -> voltage conversion -> status classification -> CSV data -> Python analysis -> plot/report evidence.

Limitation

This is simulated hardware. It does not replace real hardware evidence because real systems include noise, wiring issues, measurement error, ADC tolerance, and calibration problems.

A6. Final CSV sample

The final Wokwi data file contains 48 valid samples. A preview is shown below.

```
time_ms,adc_value,voltage,status
999,413,2.019,NORMAL
2000,500,2.444,NORMAL
3001,553,2.703,NORMAL
4002,557,2.722,NORMAL
5003,470,2.297,NORMAL
6004,432,2.111,NORMAL
7005,413,2.019,NORMAL
8006,549,2.683,NORMAL
9007,860,4.203,FAIL
10008,1023,5.000,FAIL
11009,1023,5.000,FAIL
12010,0,0.000,NORMAL
13010,95,0.464,NORMAL
14012,242,1.183,NORMAL
15012,352,1.720,NORMAL
16014,440,2.151,NORMAL
17014,530,2.590,NORMAL
18016,640,3.128,WARN
19016,701,3.426,WARN
20018,754,3.685,WARN
21018,837,4.091,FAIL
22020,940,4.594,FAIL
```